

Coding & Solution

Date	29 September 2026
Team ID	UNM123APEXLAB
Project Name	ComicCraft - AI Comic Story Creator using Gemini Models
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/25bcs032-hash/comic-craft-project
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	FastAPI, Google Gemini API (Gemini text and image models)
Key Features Implemented	1. User registration (email and password, Gmail) with confirmation email, and secure login. 2. Prompt input for theme, characters, genre, art style and tone. 3. Panel-by-panel comic story script generation using Gemini models. 4. Illustration generation for each panel with character consistency. 5. Comic viewer with captions and speech bubbles; edit or regenerate the script and single panels. 6. My Library to save comics, and download as PDF or images.
Pending / Incomplete Features	1. Sharing comics through a public link. 2. Extra automated test coverage and further input validation.
Setup / Run Instructions	1. Clone the repository and open the project folder. 2. Create and activate a virtual environment: <code>python -m venv venv</code> 3. Install dependencies: <code>pip install -r requirements.txt</code> 4. Create a <code>.env</code> file and add your Gemini API key (<code>GEMINI_API_KEY</code>). 5. Start the server: <code>uvicorn main:app --reload</code> 6. Open <code>http://127.0.0.1:8000</code> in the browser.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

ComicCraft is built with FastAPI and integrates Google Gemini models to turn a user's prompt into a complete illustrated comic. The code is organised into separate modules for authentication, story generation, image generation and comic rendering, which keeps it easy to maintain and reuse. Gemini API calls are wrapped in error handling so that failures show a clear message to the user. The project is maintained in a version control repository. Sharing comics through a link and additional test coverage are planned as future improvements.

Source Code

Repository: <https://github.com/25bcs032-hash/comic-craft-project>

Key modules of the ComicCraft application are included below.

1. API Routes (FastAPI endpoints)

```
from __future__ import annotations

from pathlib import Path
from uuid import uuid4

from fastapi import (
    APIRouter,
    Form,
    HTTPException,
    Request,
)

from fastapi.concurrency import (
    run_in_threadpool,
)

from fastapi.responses import (
    FileResponse,
    HTMLResponse,
)

from fastapi.templating import (
    Jinja2Templates,
)

from app.ai.image_generator import (
    generate_image,
)

from app.config import settings

from app.schemas import (
    ComicRequest,
    HealthResponse,
    ImageTestRequest,
)

from app.services.comic_service import (
    generate_comic,
)

router = APIRouter()

templates = Jinja2Templates(
    directory=str(settings.template_dir)
)

# -----
# HOME PAGE
# -----

@router.get(
    "/",
    response_class=HTMLResponse
)
async def home(request: Request):

    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={
            "app_name": settings.app_name,
            "text_provider": settings.text_provider,
            "image_provider": settings.image_provider,
        },
    )

# -----
# HEALTH CHECK
# -----
```

```

@router.get(
    "/health",
    response_model=HealthResponse
)
async def health() -> HealthResponse:

    return HealthResponse(
        text_provider=settings.text_provider,
        image_provider=settings.image_provider,
    )

# -----
# HTML FORM GENERATION
# -----

@router.post(
    "/generate",
    response_class=HTMLResponse
)
async def generate_from_form(
    request: Request,

    prompt: str = Form(...),

    character_name: str = Form(...),

    setting: str = Form(...),

    tone: str = Form(
        "adventurous"
    ),

    art_style: str = Form(
        "comic book"
    ),

    panel_count: int = Form(5),
):

    try:

        comic_request = ComicRequest(
            prompt=prompt,
            character_name=character_name,
            setting=setting,
            tone=tone,
            art_style=art_style,
            panel_count=panel_count,
        )

        result = await run_in_threadpool(
            generate_comic,
            comic_request
        )

        return templates.TemplateResponse(
            request=request,
            name="comic_preview.html",
            context={
                "result": result,
                "layout": result.layout,
            },
        )

    except Exception as exc:

        return templates.TemplateResponse(
            request=request,
            name="error.html",
            context={
                "message": str(exc)
            },
            status_code=500,
        )

# -----
# JSON API
# -----
@router.post(

```

```

        "/generate-comic/json"
    )
    @router.post(
        "/api/generate-comic"
    )
    async def generate_from_json(
        payload: ComicRequest
    ):
        try:
            result = await run_in_threadpool(
                generate_comic,
                payload
            )

            # Do not expose local filesystem paths
            data = result.model_dump(
                exclude={"pdf_path"}
            )

            for panel in data["layout"]:
                panel.pop(
                    "image_path",
                    None
                )

            return data

        except Exception as exc:
            raise HTTPException(
                status_code=500,
                detail=str(exc)
            ) from exc

# -----
# IMAGE TEST
# -----

@router.post(
    "/test-image"
)
async def test_image(
    payload: ImageTestRequest
):
    try:
        job_id = (
            f"test-{uuid4().hex}"
        )

        _path, url = await run_in_threadpool(
            generate_image,
            payload.prompt,
            job_id,
            1,
            payload.art_style,
        )

        return {
            "ok": True,
            "image_url": url,
            "job_id": job_id,
        }

    except Exception as exc:
        raise HTTPException(
            status_code=500,
            detail=str(exc)
        ) from exc

# -----
# PDF DOWNLOAD
# -----

@router.get(

```

```

        "/download/{filename}"
    )
    async def download_pdf(
        filename: str
    ):

        # Prevent path traversal
        safe_name = Path(filename).name

        if (
            safe_name != filename
            or not safe_name.lower().endswith(".pdf")
        ):
            raise HTTPException(
                status_code=400,
                detail="Invalid filename."
            )

        file_path = (
            settings.exports_dir
            / safe_name
        )

        if not file_path.exists():
            raise HTTPException(
                status_code=404,
                detail="PDF not found."
            )

        return FileResponse(
            path=str(file_path),
            media_type="application/pdf",
            filename=safe_name,
        )

# -----
# EXPORT SUCCESS PAGE
# -----

@router.get(
    "/export-success",
    response_class=HTMLResponse
)
async def export_success(
    request: Request,
    file: str = ""
):

    return templates.TemplateResponse(
        request=request,
        name="export_success.html",
        context={
            "file": (
                Path(file).name
                if file
                else "your comic PDF"
            )
        },
    )

```

2. Comic Outline Generation (Gemini structured output)

```
from __future__ import annotations

from app.config import settings

from app.schemas import (
    ComicRequest,
    OutlineResponse,
    PanelOutline,
)

def _mock_outline(
    request: ComicRequest,
) -> list[PanelOutline]:
    beats = [
        (
            "The Spark",
            "The adventure begins and the hero discovers the central challenge.",
        ),
        (
            "A Strange Clue",
            "A clue or unexpected event pushes the hero deeper into the mystery.",
        ),
        (
            "The Turning Point",
            "The hero faces a meaningful obstacle and must make a choice.",
        ),
        (
            "The Big Moment",
            "The main conflict reaches its most exciting moment.",
        ),
        (
            "A New Dawn",
            "The conflict resolves and the hero sees the world differently.",
        ),
        (
            "One More Surprise",
            "A final twist opens a playful new possibility.",
        ),
        (
            "The Team Grows",
            "New allies or ideas broaden the hero's journey.",
        ),
        (
            "To Be Continued",
            "A satisfying ending hints at another adventure.",
        ),
    ]

    panels: list[PanelOutline] = []

    for i in range(request.panel_count):
        title, beat = beats[i]

        scene = (
            f"{request.character_name} is in "
            f"{request.setting}. "
            f"{beat} "
            f"The mood is {request.tone}, "
            f"inspired by the idea: "
            f"{request.prompt}"
        )

        image_prompt = (
            f"{request.art_style} comic illustration, "
            f"panel {i + 1}, "
            f"{request.character_name}, "
            f"{request.setting}, "
            f"{request.tone} mood. "
            f"{beat} "
            "Clean composition, expressive character, "
            "cohesive recurring costume and appearance, "
            "no text, no watermark."
        )

        panels.append(
            PanelOutline(
```

```

        panel_number=i + 1,
        title=title,
        scene_description=scene,
        image_prompt=image_prompt,
    )
)

return panels

def generate_outline(
    request: ComicRequest,
) -> list[PanelOutline]:
    """
    Generate a structured comic outline.

    When TEXT_PROVIDER=mock:
        deterministic offline outline.

    When TEXT_PROVIDER=gemini:
        real Gemini structured generation.
    """

    # -----
    # MOCK MODE
    # -----

    if settings.text_provider != "gemini":
        return _mock_outline(request)

    # -----
    # API KEY CHECK
    # -----

    if not settings.gemini_api_key:

        raise RuntimeError(
            "TEXT_PROVIDER is set to 'gemini' "
            "but GEMINI_API_KEY is missing. "
            "Add it to .env or switch "
            "TEXT_PROVIDER=mock."
        )

    # -----
    # IMPORT GOOGLE SDK
    # -----

    try:

        from google import genai
        from google.genai import types

    except ImportError as exc:

        raise RuntimeError(
            "google-genai is not installed. "
            "Run: pip install -r requirements.txt"
        ) from exc

    # -----
    # CLIENT
    # -----

    client = genai.Client(
        api_key=settings.gemini_api_key
    )

    # -----
    # PROMPT
    # -----

    prompt = f"""
You are the outline writer for ComicCraft,
a family-friendly comic story creator.

Create exactly {request.panel_count}
sequential comic panels.

User idea:
{request.prompt}
Main character:

```

```
{request.character_name}
```

Setting:

```
{request.setting}
```

Tone:

```
{request.tone}
```

Art style:

```
{request.art_style}
```

For every panel provide:

- panel_number starting at 1
- a short title
- a vivid scene_description that advances one coherent story
- an image_prompt that visually describes the scene
- recurring character appearance
- camera/composition
- mood
- requested art style

Keep violence non-graphic and avoid sexual content.

Image prompts must not include:

- speech bubbles
- written captions
- logos
- watermarks

Text will be rendered separately by the ComicCraft application.

```
"".strip()
```

```
# -----
# GEMINI REQUEST
# -----

response = client.models.generate_content(
    model=settings.gemini_outline_model,
    contents=prompt,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=OutlineResponse,
        temperature=0.8,
    ),
)

if not response.text:

    raise RuntimeError(
        "Gemini returned an empty "
        "outline response."
    )

# -----
# VALIDATE JSON
# -----

parsed = OutlineResponse.model_validate_json(
    response.text
)

# -----
# PANEL COUNT VALIDATION
# -----

if len(parsed.panels) != request.panel_count:

    raise RuntimeError(
        f"Gemini returned "
        f"{len(parsed.panels)} panels; "
        f"expected {request.panel_count}."
    )

return parsed.panels
```

3. Panel Story / Script Generation (Gemini structured output)

```
from __future__ import annotations

from app.config import settings

from app.schemas import (
    ComicRequest,
    PanelOutline,
    PanelStory,
    StoryResponse,
)

def _mock_story(
    request: ComicRequest,
    outline: list[PanelOutline],
) -> list[PanelStory]:

    stories: list[PanelStory] = []

    for panel in outline:

        stories.append(
            PanelStory(
                panel_number=panel.panel_number,

                caption=(
                    f'{request.setting} - '
                    f'panel {panel.panel_number}.'
                ),

                narration=(
                    f'{request.character_name} '
                    f'moves through the moment '
                    f'with a {request.tone} energy. '
                    f'{panel.scene_description}'
                ),

                dialogue=(
                    f'{request.character_name}: '
                    f'"I have to see where this leads!"'
                    if panel.panel_number
                    < len(outline)
                    else
                    f'{request.character_name}: '
                    f'"What an adventure."'
                ),
            )
        )

    return stories

def generate_story(
    request: ComicRequest,
    outline: list[PanelOutline],
) -> list[PanelStory]:

    """
    Expand comic outline into:
    - captions
    - narration
    - dialogue
    """

    # -----
    # MOCK
    # -----

    if settings.text_provider != "gemini":

        return _mock_story(
            request,
            outline
        )

    # -----
    # API KEY
    # -----
    if not settings.gemini_api_key:
```

```

        raise RuntimeError(
            "TEXT_PROVIDER is set to 'gemini' "
            "but GEMINI_API_KEY is missing."
        )

# -----
# IMPORT
# -----

try:

    from google import genai
    from google.genai import types

except ImportError as exc:

    raise RuntimeError(
        "google-genai is not installed. "
        "Run: pip install -r requirements.txt"
    ) from exc

# -----
# PREPARE OUTLINE
# -----

outline_json = [
    item.model_dump()
    for item in outline
]

# -----
# PROMPT
# -----

prompt = f"""
You are the script writer for ComicCraft.

Expand the supplied comic outline into
polished, family-friendly comic writing.

Preserve the exact panel order and
panel numbers.

Main character:
{request.character_name}

Setting:
{request.setting}

Tone:
{request.tone}

Original idea:
{request.prompt}

OUTLINE:

{outline_json}

For every panel return:

- panel_number
- caption:
  one short atmospheric caption
- narration:
  1-3 concise sentences describing
  action and emotion
- dialogue:
  natural character dialogue

Keep dialogue short enough
to fit a comic panel.

Do not add panels.

Do not remove panels.

Avoid graphic violence
and sexual content.
""".strip()
# -----

```

```

# GEMINI
# -----

client = genai.Client(
    api_key=settings.gemini_api_key
)

response = client.models.generate_content(
    model=settings.gemini_story_model,
    contents=prompt,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=StoryResponse,
        temperature=0.9,
    ),
)

if not response.text:

    raise RuntimeError(
        "Gemini returned an empty "
        "story response."
    )

# -----
# VALIDATE
# -----

parsed = StoryResponse.model_validate_json(
    response.text
)

if len(parsed.panels) != len(outline):

    raise RuntimeError(
        f"Gemini returned "
        f"{len(parsed.panels)} story panels; "
        f"expected {len(outline)}."
    )

return parsed.panels

```